AI Engineering and Claude Architect · Part V, Claude Code

Claude Code, Lesson 2

CLAUDE.md, Rules and Memory

A rule file used `globs:` where the field is `paths:`. There was no error. The rule simply loaded in every session, for months. Most configuration mistakes here fail silently, so you have to check what actually loaded.

Module	AI12 of 13
Lesson	2 of 5
Prerequisites	AI12 L1, AI03 L6
Exam relevance	**Core: TS 3.1, 3.3.** Also TS 2.4
Reading time	~30 minutes

Contents

1. CLAUDE.md, Rules and Memory

Lesson 2 of 5 · Claude Code

CLAUDE.md, Rules and Memory

Key idea

Scope decides who gets an instruction, and path rules decide when it loads, while neither makes it mandatory. Team standards go in project files, personal ones in user files, and spread-out conventions in glob rules. Keep files short, and use `/memory` to see what's really loaded. What must always hold belongs in a hook.

> *Before: the loop, context engineering. Teaches: the hierarchy, how files load, imports, `.claude/rules/`, path scoping, what to write, auto memory, and diagnosing silent failures. Used by: L3, L5, AI13.*

The hook: nothing tells you

I wrote a linter for mistakes that the docs say produce no error, and ran it on a sample project with deliberate errors. Real output from `ConfigLint.java`:

```
A sample project with deliberate mistakes:
  .claude/agents/reader.md: field 'allowedTools' is ignored without an error, so the restriction you meant doesn't
apply
  .claude/rules/spec.md: glob **/*.spec.ts matches no file here
  .claude/rules/tests.md: field 'globs' is ignored, as only paths is read. The rule loads in every session
  .claude/skills/analyze/SKILL.md: no description, so Claude can't tell when it applies
  .claude/skills/analyze/SKILL.md: note: allowed-tools pre-approves tools for that turn. It doesn't block others, so
use deny rules for that
  .claude/skills/analyze/SKILL.md: unknown field 'contexts' in frontmatter
  .mcp.json: looks like a committed secret. Use ${VAR} expansion
  CLAUDE.local.md: personal file not in .gitignore, so it may be committed
  CLAUDE.md: 264 lines. The docs target under 200, so move parts to .claude/rules/ or skills
  CLAUDE.md: 3 emphatic rules. If they must always hold, use hooks or deny rules, since this file is only context
```

None of these stops Claude Code from starting. **Configuration that's wrong looks exactly like configuration that's working, until behavior drifts.** The memory docs say that `paths` is the only field read from a rule, and any other is ignored without an error.

This linter is my own small sketch, and its checks come from statements in the docs. It isn't an Anthropic tool. `/doctor` is the built-in one for some of this.

Intuition: office notices

An office has a company handbook, a notice on your floor's wall, a note taped to the printer, and a sticky note on your own monitor. Each reaches different people at different moments. The sticky note helps only you, and a new joiner will never see it. The printer note appears only when you're at the printer, whichever floor it's on.

Those are user files, project files and path rules. All of them are notices, though, and a notice isn't a locked door.

Mechanism

Scope: who gets it

From the memory docs, in load order, broadest first:

Scope	Location	Shared with	Use for
Managed policy	System path set by IT	Everyone in the organization	Company standards
User	`~/.claude/CLAUDE.md`	Only you, in all projects	Personal preferences
Project	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Team, through Git	Architecture, commands, standards
Local	`./CLAUDE.local.md`	Only you, in this project	Sandbox URLs. Add to `.gitignore`
Directory	`CLAUDE.md` in a subdirectory	Team, through Git	One package's conventions

The classic diagnosis comes straight from the guide: a new teammate doesn't get a standard, because it was in someone's `~/.claude/CLAUDE.md`. **If a teammate needs it, it has to be in Git.**

How files load

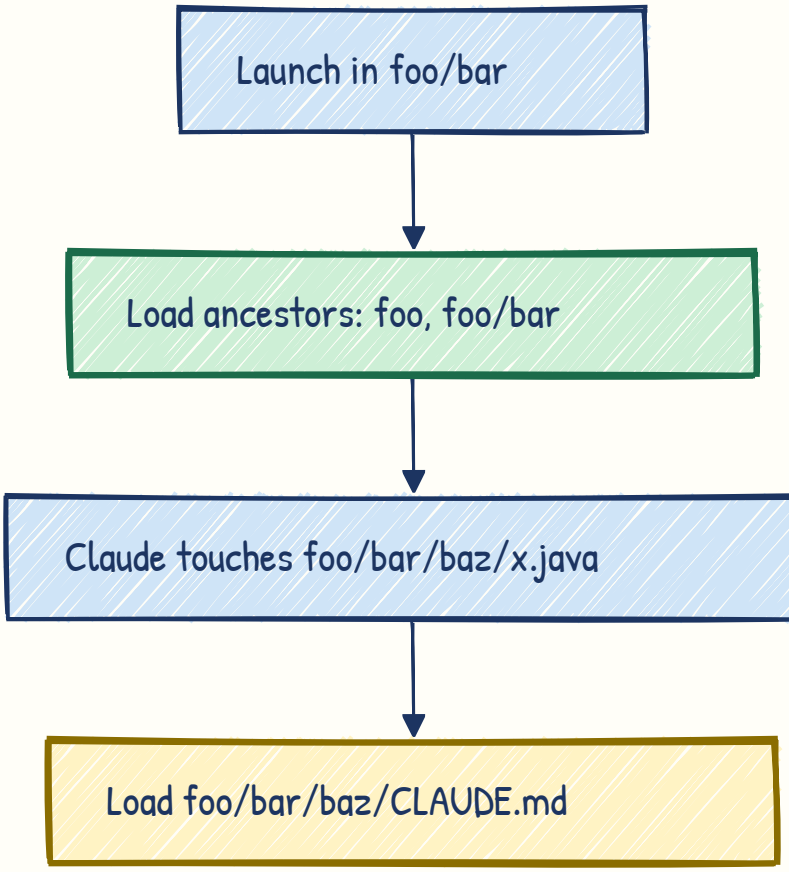


Figure 1. Files above you load at launch, and files below load when work reaches them. All of it is added to context, and nothing overrides anything.

Three facts people get wrong, from the docs:

Fact	Consequence
Discovered files are **concatenated, not overridden**	"Closer" text is read last, and it doesn't erase earlier text
If two instructions contradict, Claude **may pick one arbitrarily**	Inconsistent behavior, so remove the conflict
Subdirectory files load **on demand**, when Claude reads or edits there	Absent until then, and they may need to reload after `/compact`

After `/compact`: the project-root CLAUDE.md is re-read from disk. Nested files and path-scoped rules load again only when a matching file is touched. Instructions given only in conversation are lost, which is a common cause of "it forgot."

Imports

`@path/to/import` pulls another file in, and the guide calls this `@import`. Relative paths resolve from the file containing the import, not the working directory. Imports can nest, up to four hops deep. Text inside backticks isn't imported.

The guide's skill: each package's CLAUDE.md imports only the standards that its maintainers choose.

```
# packages/billing/CLAUDE.md
Money handling: @../../docs/standards/money.md
API errors: @../../docs/standards/errors.md
```

Imports still load into context. They organize text, and don't save tokens. Path rules and skills save tokens.

`.claude/rules/`

Topic files in place of one long CLAUDE.md, such as `testing.md` and `api-design.md`. All `.md` files are found recursively. Rules without `paths` load at launch, at the same priority as `.claude/CLAUDE.md`.

Path-scoped rules

```
---
paths:
  - "src/api/**/*.ts"
---
# API rules
- All endpoints must validate input
```

They load when Claude works with matching files.

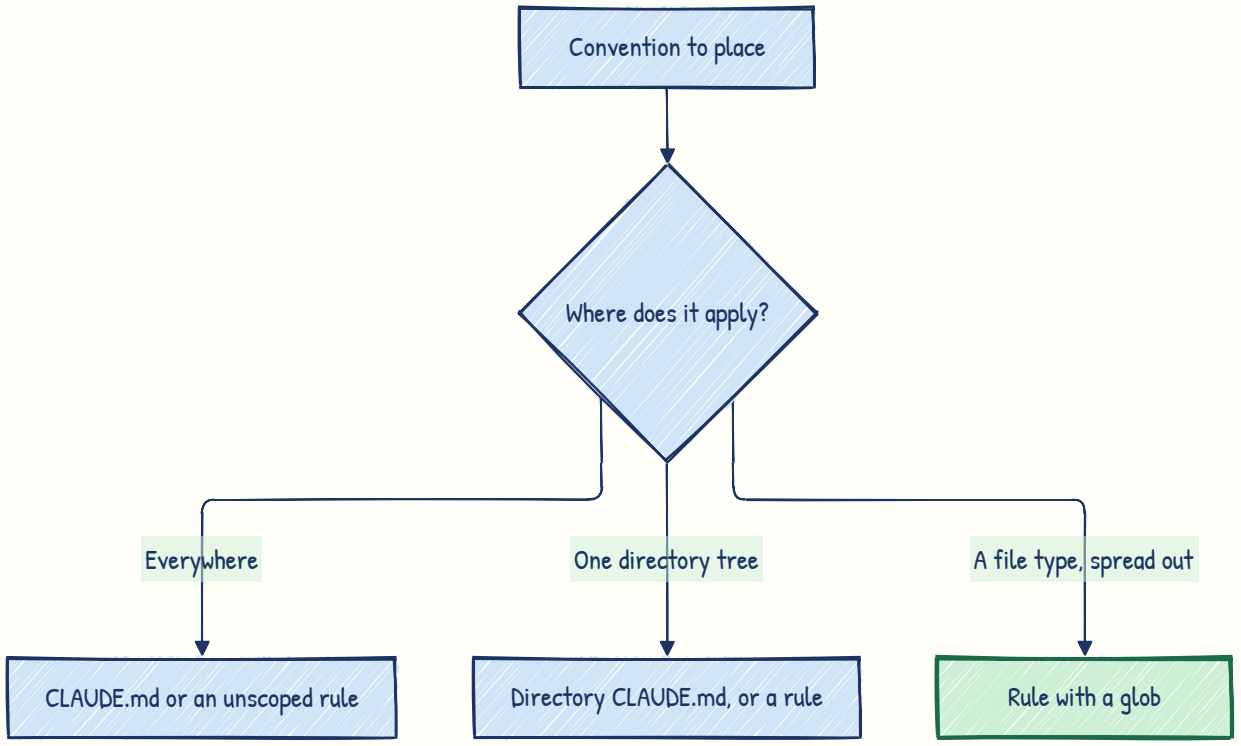


Figure 2. A glob rule follows the file type, wherever it lives. A directory CLAUDE.md follows one location.

Test files beside their source are the standard example: `**/*.test.tsx` covers them all, in one file, while a directory CLAUDE.md would need a copy in every folder.

Silent failures, from the docs:

Mistake	What happens
A field other than `paths`	Ignored, and the rule behaves as unscoped
YAML that doesn't parse	Frontmatter ignored, and the rule loads as if it had no `paths`
A glob that matches nothing	Never loads, and nothing warns you

Run `claude --debug` to see parse errors. And note my linter's `no file here` check uses Java's glob matcher, which approximates Claude Code's.

What to write

The best practices page puts it well: every line takes a little attention on every task. It should hold what Claude can't work out alone:

Include	Leave out
Exact test, typecheck and lint commands	What the code already shows
Misleading areas: "`src/legacy/` is live, don't copy it"	Generic advice: "write clean code"
Decisions that aren't in the repo	Long procedures: use a skill (L3)
Things you've corrected twice	Rules for one area: use a path rule

From the memory docs, on wording: "Use 2-space indentation" instead of "Format code properly". Target **under 200 lines** per file, as longer files consume more context and reduce adherence.

Hook or line? The docs give a simple test: "If you keep adding emphasis to a `CLAUDE.md` line and Claude still misses it, that's a sign the line belongs in a hook." The reverse holds too: delete hooks that only express preferences.

Auto memory

Claude also writes its own notes from your corrections. From the docs: it's per repository, shared across worktrees, and the first 200 lines or 25 KB load at each session. It's separate from CLAUDE.md, which you write. The guide predates much of this, so the exam concentrates on CLAUDE.md.

Diagnosing

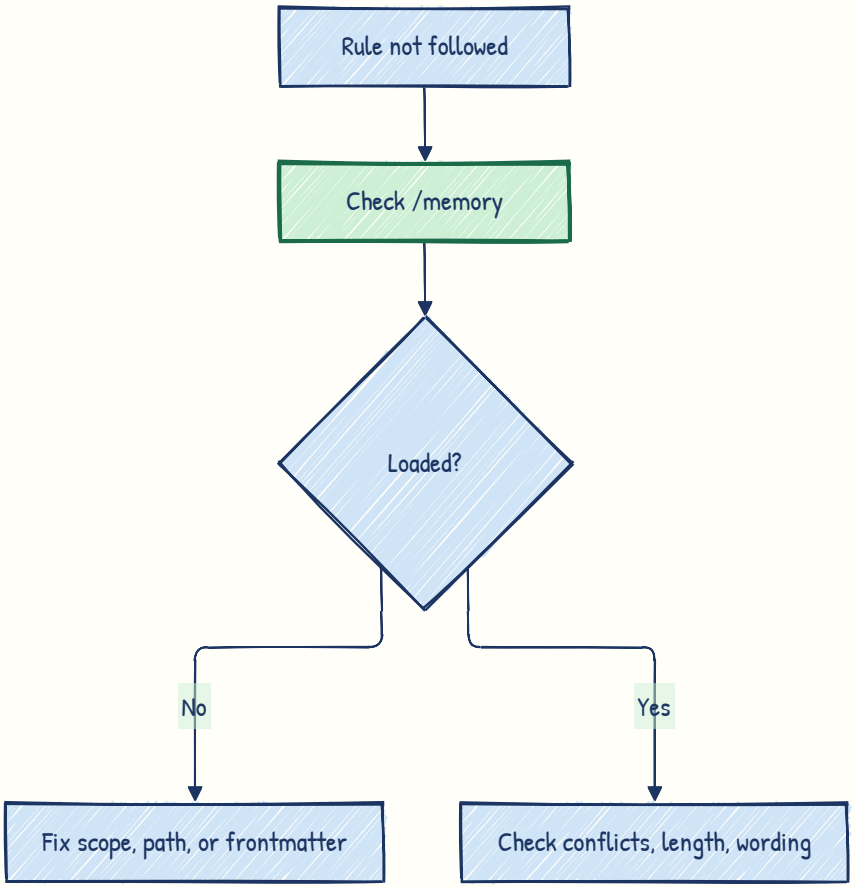


Figure 3. Find out what's loaded before changing the wording. Most 'it ignores my rule' problems are loading or conflict problems.

`/memory` lists loaded files. The best practices page also mentions `/context all`, to confirm that a file loaded, and `/doctor prompt-audit`, which flags instructions that contradict each other or point at missing files.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Put team rules in `~/.claude/CLAUDE.md`	Teammates never get them
A CLAUDE.md copy in each test folder	Copies drift, so use one glob rule
One huge root CLAUDE.md	Costs attention on every task
Add NEVER in capitals	Still advice, so use a hook
Rely on the closest file overriding	Files concatenate, and don't override
Imports, to save context	They load anyway

How Anthropic solved it

Mechanism	What's published	Source
Scope levels	Managed, user, project, local	Memory docs
Load behavior	Ancestors at launch, subdirectories on demand	Memory docs
Rules	`.claude/rules/` , `paths`	Memory docs
Guidance	Short, concrete, under 200 lines	Memory docs and best practices
Determinism split	Prompt-level or harness-level	Best practices
Tooling	`/memory` , `/context` , `/init` , `/doctor`	Docs

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: one root CLAUDE.md, growing without limit.

Improved: split into topics and paths. New problem: nobody notices when a rule stops loading, or when two files contradict each other.

Production: treat configuration as code, reviewed and checked in CI.

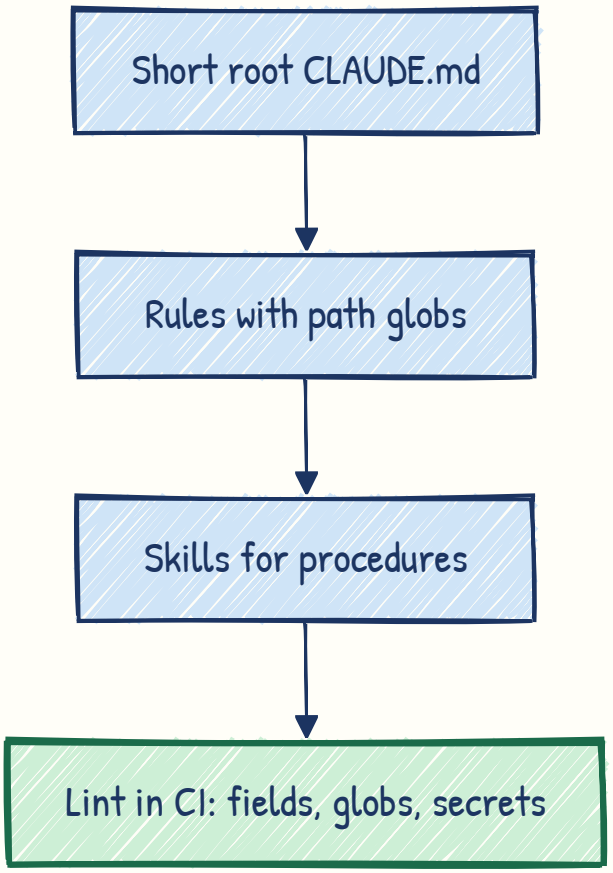


Figure 4. A short root, scoped rules and skills. CI lints what would otherwise fail silently.

Component	Why it exists	How it communicates
Root CLAUDE.md	Commands and gotchas	Loaded at launch
Path rules	Conventions for file types	Load on match
Package files with imports	Maintainers choose standards	Load on demand
Local or user files	Personal preferences stay out of Git	Loaded for that person
Code review	Config is reviewed like code	Pull requests
Config lint	Catches silent mistakes	CI step

Failures:

Failure	How you'd notice	What to do
New hire misses standards	Their output differs	Move to project scope
Rule never loads	`/memory`, or its glob matches nothing	Fix the glob or field
Rule loads everywhere	Context grows	Fix `paths`, as `globs:` is ignored
Contradictions	Inconsistent behavior	Remove one
Forgotten after `/compact`	Conversation-only instructions vanish	Put them in a file
Stale instructions	Claude acts on old facts	Audit regularly
Other teams' files load in a monorepo	Odd rules appear	`claudeMdExcludes`, per the docs

Java

The linter's rule check is the whole idea: a field that isn't `paths` is worth reporting, because Claude Code won't.

```
if (!key.equals("paths")) {
```

In your own CI, this is a small script. It reads frontmatter and globs, and reports mismatches. I ran it on this course's own repository, and it reported nothing, which only means that none of these checks fired:

```
This course's own repository:
  no findings
```

Trade-offs

Decision	Benefit	Cost
Short CLAUDE.md	Better adherence, less context	Details must live elsewhere
Path rules	Load when relevant	Silent failure if mis-scoped
Directory files	Natural for one package	Copies for spread-out conventions
Imports	Modular and selective	Still load, and can nest
Auto memory	Learns corrections	Less reviewable than a file you wrote

When not to bother: a small project needs one short CLAUDE.md, and nothing else.

Common mistakes

Testing a rule by asking Claude whether it knows it. Use `/memory`. The model's answer isn't evidence of what loaded.

Common mistakes

Thinking a scoped rule is a security control. Rules are context. A deny rule or a hook is what prevents an action.

What to say in an interview

"Scope decides who gets an instruction: user files are personal, project files are shared through Git. Path rules load by glob, which suits test files spread across a repo. It's all context and not enforcement, and files concatenate rather than override, so contradictions make behavior arbitrary. I keep files short, use `/memory` to see what loaded, and lint for silent mistakes like a wrong frontmatter field."

Certification lens

How this lesson maps to CCAR-F

TS 3.1 and TS 3.3 are exam content: the hierarchy, `@import`, `.claude/rules/`, `/memory`, `paths` globs, and rules versus directory files. Current docs write imports as `@path`, and add local files and auto memory, which the guide doesn't cover.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Code Generation · Easy. A personal preference for terse explanations should apply in all your projects, and not bother colleagues. Where does it go?

A. The project's root `CLAUDE.md`

B. `~/.claude/CLAUDE.md`

C. `.claude/rules/style.md`

D. `.mcp.json`

A **B** (TS 3.1). User level is personal, across projects. A and C are shared with the team, and D configures servers.

Q Q2 · Exam style, scenario: Code Generation · Medium. All migration files, in many directories, need one convention. What's the best approach?

A. A `CLAUDE.md` in each directory containing migrations

B. A section in the root `CLAUDE.md`

C. A skill that developers run before writing migrations

D. One rule file with a `paths` glob matching migration files

A **D** (TS 3.3). One glob rule loads when relevant. A drifts, B loads for all work, and C relies on people remembering.

Q Q3 · Exam style, scenario: Code Generation · Medium. Claude follows the team's API standard on some machines only. What's the most direct first diagnostic?

A. Run `/memory` on an affected machine to see which files are loaded

B. Rewrite the rule in capital letters

C. Reinstall Claude Code on all of the machines

D. Ask Claude to explain the standard from its memory

A **A** (TS 3.1). It shows what's loaded. B assumes that wording is the issue, C is unrelated, and D isn't evidence of what loaded.

Q Q4 · Exam style, scenario: Code Generation · Hard. A 1,200-line CLAUDE.md causes inconsistent results, and most sections matter only for certain packages. What's the best first restructuring?

A. Move it all to user-level files, so that it stops being shared

B. Keep it, and add a table of contents at the top

C. Split it into topic rules and package imports, leaving a short root file

D. Convert every section into hooks

A C (TS 3.1). It's modular, and loads what's relevant. A loses sharing, B doesn't reduce load, and hooks can't express most conventions.

Q Q5 · A rule's frontmatter has `include: ["**/*.sql"]`. Sessions on unrelated files seem to have it. Why?

A `paths` is the only field read, per the docs. `include` is ignored without an error, so the rule behaves as unscoped, and it loads at launch. Rename the field, and check with `/memory`.

Revision - Lesson 2

→ Scope: managed, user, project, local, directory. User-level isn't shared with teammates.

→ Ancestors load at launch, and subdirectory files load on demand.

→ Files concatenate and don't override. Contradictions can resolve arbitrarily.

→ `@import` (`@path`) makes files modular, up to four hops. Imported text still loads.

→ `.claude/rules/` holds topic files. `paths` globs load them for matching files.

→ Only `paths` is read. Other fields and bad YAML fail silently into always-loaded.

→ Prefer glob rules for spread-out conventions, and directory files for one tree.

→ Keep CLAUDE.md under 200 lines. Include what Claude can't work out.

→ Everything here is context. If it must always hold, use a hook.

→ Diagnose with `/memory` first. After `/compact`, nested files reload when touched.

Your notes